



Faculty of Computer Applications

**MCA Sem-1
(AY-2026-27)**

Lab Book

**Sub Code: 05MC0108
(DBMS)**

Enrollment No :	92600584037
Full Name :	Darshit Mehta

Subject Faculty Name: Prof. Amit Gohel

Index

Unit : 1 Exercise No	Unit : 1 Definition
1	Write a PL/SQL block that find 1. Area of Circle 2. Area of Rectangle 3. Circumference of Circle
2	Write a PL/SQL block to accept product name, qty and price from user and then calculate discount in Rs. based on the given (%).
3	Write a PL/SQL block to calculate the total, percentage and grade of student based on his/her Rollno from RESULT table. (Create RESULT table with Rollno, Name, Sub1, Sub2, Sub3, Sub4, Sub5, Total, Per, Grade attributes with appropriate data type).
4	1. Write a program that prints value 1 to 100 numbers using FOR LOOP. 2. Write a program that prints value 1 to 100 number using LOOP Command. 3. Write a program that prints value 1 to 100 number using WHILE LOOP.
5	Write a PL/SQL block which displays gross salary of employees as per user input EID. (Consider EMP table with EID, EName, Deptno, Deptname Gender, Age, BasicSal) with appropriate data types.) Gross_Salary: BASICSAL + (DA + HRA + Medical) – PF. Rules: HRA = 15% of basic, DA = 50% of basic, Medical = Rs. 500, PF = 10% of basic.
6	Write a PL/SQL block which accepts measurement in feet and displays it in cm, inch and meter.
7	Write a PL/SQL block which converts temperature from Celsius to Fahrenheit.
8	Write a PL/SQL block to delete the record of employee for given EID.
9	Write a PL/SQL block that calculates the simple interest based on given principal amount, rate of interest and number of years.
10	Write a PL/SQL block to calculate the square and cube of the given number.

Unit : 2 Exercise No	Unit : 2 Definition
11	Write a simple procedure that increases by the salary of employees for the given department no. by percentage inputted by the user using IN parameter. Also, handle the exception if inputted department number not found.
12	Write a procedure that search whether the given employee number is present or not in the table if present then print name, salary and department number of that employee. (Use both IN and OUT mode variables) and Write a PL/SQL block to call the SEARCH_EMP procedure.
13	Write a procedure that accept employee ID and remove records from employee table. Also, create a calling program to call procedure.
14	Write a function that returns the age category of an employee. Categories: Age < 18 -> Minor 18 to 60 -> Adult Age > 60 -> Senior
15	Write a PACKAGE which includes Package Declaration, Package Body and call the individual object from the Package.
Unit : 3 Exercise No	Unit : 3 Definition
16	Write a trigger to insert the existing values of the EMP table into NEWEMP table when the record is deleted from EMP table.
17	Write a trigger to insert the existing values of the EMP table into NEWEMP table when the record is updated in EMP table.
18	Write a trigger to insert the values into the NEWEMP table when the records are inserted into the EMP table.
19	Write a trigger that restricts the entry of record if salary is greater than Rs.50000.

20	Write a trigger to restrict user form using the table on Sunday.																																																																																																																
Unit : 5 Exercise No	Unit : 5 Definition																																																																																																																
21	Create a database with name emp_data & perform following 1. Create collection : employee & insert data as per given table																																																																																																																
	<table><tr><th>EMPNO</th><th>EMPNAME</th><th>JOB</th><th>MGR</th><th>HIREDATE</th><th>SAL</th><th>COMM.</th><th>DEPTNO</th></tr><tr><td>7369</td><td>SMITH</td><td>CLERK</td><td>7902</td><td>17-DEC-00</td><td>8000</td><td></td><td>20</td></tr><tr><td>7499</td><td>ALLEN</td><td>SALESMAN</td><td>7698</td><td>20-FEB-01</td><td>16000</td><td>300</td><td>30</td></tr><tr><td>7521</td><td>WARD</td><td>SALESMAN</td><td>7698</td><td>22-FEB-01</td><td>12500</td><td>500</td><td>30</td></tr><tr><td>7566</td><td>JONES</td><td>MANAGER</td><td>7839</td><td>02-APR-01</td><td>29750</td><td></td><td>20</td></tr><tr><td>7654</td><td>MARTIN</td><td>SALESMAN</td><td>7698</td><td>20-SEP-01</td><td>12500</td><td>1400</td><td>30</td></tr><tr><td>7698</td><td>BLAKE</td><td>MANAGER</td><td>7839</td><td>01-MAY-01</td><td>28500</td><td></td><td>30</td></tr><tr><td>7782</td><td>CLARK</td><td>MANAGER</td><td>7839</td><td>09-JUN-01</td><td>24500</td><td></td><td>10</td></tr><tr><td>7788</td><td>SCOTT</td><td>ANALYST</td><td>7566</td><td>19-APR-07</td><td>30000</td><td></td><td>20</td></tr><tr><td>7839</td><td>KING</td><td>PRESIDENT</td><td></td><td>17-NOV-01</td><td>50000</td><td></td><td>10</td></tr><tr><td>7844</td><td>TURNER</td><td>SALESMAN</td><td>7698</td><td>08-SEP-01</td><td>15000</td><td></td><td>30</td></tr><tr><td>7876</td><td>ADAMS</td><td>CLERK</td><td>7788</td><td>23-MAY-07</td><td>11000</td><td></td><td>20</td></tr><tr><td>7900</td><td>JAMES</td><td>CLERK</td><td>7698</td><td>03-DEC-01</td><td>9500</td><td></td><td>30</td></tr><tr><td>7902</td><td>FORD</td><td>ANALYST</td><td>7566</td><td>03-DEC-01</td><td>30000</td><td></td><td>20</td></tr></table>	EMPNO	EMPNAME	JOB	MGR	HIREDATE	SAL	COMM.	DEPTNO	7369	SMITH	CLERK	7902	17-DEC-00	8000		20	7499	ALLEN	SALESMAN	7698	20-FEB-01	16000	300	30	7521	WARD	SALESMAN	7698	22-FEB-01	12500	500	30	7566	JONES	MANAGER	7839	02-APR-01	29750		20	7654	MARTIN	SALESMAN	7698	20-SEP-01	12500	1400	30	7698	BLAKE	MANAGER	7839	01-MAY-01	28500		30	7782	CLARK	MANAGER	7839	09-JUN-01	24500		10	7788	SCOTT	ANALYST	7566	19-APR-07	30000		20	7839	KING	PRESIDENT		17-NOV-01	50000		10	7844	TURNER	SALESMAN	7698	08-SEP-01	15000		30	7876	ADAMS	CLERK	7788	23-MAY-07	11000		20	7900	JAMES	CLERK	7698	03-DEC-01	9500		30	7902	FORD	ANALYST	7566	03-DEC-01	30000		20
	EMPNO	EMPNAME	JOB	MGR	HIREDATE	SAL	COMM.	DEPTNO																																																																																																									
	7369	SMITH	CLERK	7902	17-DEC-00	8000		20																																																																																																									
	7499	ALLEN	SALESMAN	7698	20-FEB-01	16000	300	30																																																																																																									
	7521	WARD	SALESMAN	7698	22-FEB-01	12500	500	30																																																																																																									
	7566	JONES	MANAGER	7839	02-APR-01	29750		20																																																																																																									
	7654	MARTIN	SALESMAN	7698	20-SEP-01	12500	1400	30																																																																																																									
	7698	BLAKE	MANAGER	7839	01-MAY-01	28500		30																																																																																																									
	7782	CLARK	MANAGER	7839	09-JUN-01	24500		10																																																																																																									
	7788	SCOTT	ANALYST	7566	19-APR-07	30000		20																																																																																																									
	7839	KING	PRESIDENT		17-NOV-01	50000		10																																																																																																									
	7844	TURNER	SALESMAN	7698	08-SEP-01	15000		30																																																																																																									
	7876	ADAMS	CLERK	7788	23-MAY-07	11000		20																																																																																																									
	7900	JAMES	CLERK	7698	03-DEC-01	9500		30																																																																																																									
	7902	FORD	ANALYST	7566	03-DEC-01	30000		20																																																																																																									
	2. Create collection: Department & insert data as per given table.																																																																																																																
	<table><tr><th>DEPTNO</th><th>DNAME</th><th>LOC</th></tr><tr><td>10</td><td>ACCOUNTING</td><td>NEW YORK</td></tr><tr><td>20</td><td>RESEARCH</td><td>DALLAS</td></tr><tr><td>30</td><td>SALES</td><td>CHICAGO</td></tr><tr><td>40</td><td>OPERATING</td><td>BOSTON</td></tr></table>	DEPTNO	DNAME	LOC	10	ACCOUNTING	NEW YORK	20	RESEARCH	DALLAS	30	SALES	CHICAGO	40	OPERATING	BOSTON																																																																																																	
	DEPTNO	DNAME	LOC																																																																																																														
	10	ACCOUNTING	NEW YORK																																																																																																														
	20	RESEARCH	DALLAS																																																																																																														
30	SALES	CHICAGO																																																																																																															
40	OPERATING	BOSTON																																																																																																															
3. Show all databases.																																																																																																																	
4. Show currently selected database.																																																																																																																	
5. Display all employee records.																																																																																																																	
6. Display details of employees working as CLERK.																																																																																																																	
7. Retrieve employees belonging to department number 30.																																																																																																																	
8. Find employees whose salary is greater than 20000.																																																																																																																	
9. Display employees whose job is SALESMAN.																																																																																																																	
10. Retrieve employees who do not belong to department 10.																																																																																																																	
11. Display employees with salary between 15000 and 30000.																																																																																																																	
12. Find employees whose commission is not null.																																																																																																																	
13. Display employees whose job is MANAGER and department is 20.																																																																																																																	
14. Retrieve employees hired after 01-JAN-01.																																																																																																																	
15. Display employees hired before 01-JAN-01.																																																																																																																	
16. Find employees whose salary is less than or equal to 10000.																																																																																																																	
17. Display employees whose name starts with 'S'.																																																																																																																	
18. Find employees whose name contains 'AR'.																																																																																																																	
19. Retrieve employees working under manager 7698.																																																																																																																	
20. Display employees whose job is ANALYST or CLERK.																																																																																																																	
21. Find employees who belong to either department 10 or 20.																																																																																																																	
Display only empno, empname, and salary of all employees.																																																																																																																	
22	Prepare following collection & solve queries:																																																																																																																

Roll No	Name	Branch	Semester	Percentage	Grade
1	Amit	CSE	4	78	B
2	Neha	IT	4	85	A
3	Rahul	CSE	3	65	C
4	Priya	ECE	4	92	A
5	Karan	IT	2	55	D
6	Anita	CSE	1	88	A
7	Rakesh	ECE	3	72	B
8	Pooja	IT	4	90	A
9	Arjun	CSE	2	60	C
10	Sneha	ECE	1	82	B

1. Display all documents from the student collection.
2. Retrieve details of students belonging to the IT branch.
3. Find students whose percentage is greater than 75.
4. Display students who are in semester 3.
5. Retrieve students having grade = 'A'.
6. Display only name and branch of all students.
7. Find students whose percentage is less than 60.
8. Display students from CSE branch and semester 4.
9. Retrieve students whose percentage lies between 65 and 85.
10. Find students who are not from ECE branch.
11. Display student records sorted by percentage in ascending order.
12. Display student records sorted by name in descending order.
13. Retrieve top 5 students based on percentage.
14. Find students whose name starts with letter 'A'.
15. Find students whose name contains 'ra'.
16. Display students who belong to either CSE or IT branch.
17. Retrieve students who are neither in semester 1 nor semester 2.
18. Display only roll number, name, and percentage (exclude _id).
19. Find students whose percentage is greater than or equal to 90.

Retrieve students whose grade is not equal to 'D'.

23

1. Create a database named "mydb"
2. Show all databases.
3. Show currently selected database.
4. Create a collection named "emp".
5. Show all collections of selected database.
6. Insert following documents in emp collection:

Empno	Ename	Sal	Desig	Dept
1	Sachin	60000	Manager	Purchase
2	Kohli	50000	Manager	Sales
3	Dhoni	30000	Clerk	Admin
4	Dravid	40000	Manager	Purchase
5	Bumrah	50000	Manager	Sales
6	Jadeja	20000	Clerk	Admin
7	Ashwin	45000	Manager	Sales

Find()

1. List all documents.
2. List all documents with formatted output.
3. List the document of employee whose name is "achin"
4. List the documents of employee whose salary is less than 30000
5. List employees whose designation is manager and
6. department is sales.
7. List employees whose salary is less than 50000 and designation is manager or department is admin.
8. Arrange the records by name in descending order.
9. List first 3 documents of emp collection.
10. Skip first 3 documents of emp collection.
11. List 3rd and 4th documents of emp collection.
12. Count no. of employees.
13. List distinct designation.

Aggregate()

1. Calculate annual salary of employees
2. Display no. of employees designation wise.
3. Display total salary of employees department wise.
4. Display employee whose salary is highest.
5. List highest salary of the employee department wise.

Calculate total salary of employees department wise and list whose total is greater than 70000.

1. Write a PL/SQL block that find:

1. Area of Circle
 2. Area of Rectangle
 3. Circumference of Circle
-

```
/*
```

```
Write a PL/SQL block that find
```

- 1 . Area Of Circle
- 2 . Area Of Rectangle
3. Circumference of Circle

```
*/
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  r NUMBER;
```

```
  l NUMBER;
```

```
  b NUMBER;
```

```
  area_circle NUMBER;
```

```
  area_rectangle NUMBER;
```

```
  circumference NUMBER;
```

```
BEGIN
```

```
  r := &radius;
```

```
  l := &length;
```

```
  b := &breadth;
```

```
  area_circle := 3.14 * r * r;
```

```
  area_rectangle := l * b;
```

```
  circumference := 2 * 3.14 * r;
```

```
  DBMS_OUTPUT.PUT_LINE('Area of Circle = ' || area_circle);
```

```
  DBMS_OUTPUT.PUT_LINE('Area of Rectangle = ' || area_rectangle);
```

```
  DBMS_OUTPUT.PUT_LINE('Circumference of Circle = ' || circumference);
```

```
END;
```

```
/SET SERVEROUTPUT OFF
```

```
SET FEEDBACK ON
```

```
SET VERIFY ON
```

Output:

```
Enter value for radius: 5
old 9:      r := &radius;
new 9:      r := 5;
Enter value for length: 4
old 10:     l := &length;
new 10:     l := 4;
Enter value for breadth: 3
old 11:     b := &breadth;
new 11:     b := 3;
Area of Circle = 78.5
Area of Rectangle = 12
Circumference of Circle = 31.4

PL/SQL procedure successfully completed.
```

2. Write a PL/SQL block to accept product name, qty and price from user and then calculate discount in Rs. based on the given (%).

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    pname VARCHAR2(30);
```

```
    qty NUMBER;
```

```
    price NUMBER;
```

```
    discount_per NUMBER;
```

```
    total NUMBER;
```

```
    discount_rs NUMBER;
```

```
    net_amount NUMBER;
```

```
BEGIN
```

```
    pname := '&Product_Name';
```

```
    qty := &Quantity;
```

```
    price := &Price;
```

```
    discount_per := &Discount_Percentage;
```

```
    total := qty * price;
```

```
    discount_rs := (total * discount_per) / 100;
```

```
    net_amount := total - discount_rs;
```

```
    DBMS_OUTPUT.PUT_LINE('Product Name    : ' || pname);
```

```
    DBMS_OUTPUT.PUT_LINE('Quantity      : ' || qty);
```

```
    DBMS_OUTPUT.PUT_LINE('Price         : ' || price);
```

```
    DBMS_OUTPUT.PUT_LINE('Total Amount  : ' || total);
```

```
    DBMS_OUTPUT.PUT_LINE('Discount (Rs.) : ' || discount_rs);
```

```
    DBMS_OUTPUT.PUT_LINE('Net Amount    : ' || net_amount);
```

```
END;
```

```
/
```

Output:

```
Enter value for product_name: bat
old 10:      pname := '&Product_Name';
new 10:      pname := 'bat';
Enter value for quantity: 50
old 11:      qty := &Quantity;
new 11:      qty := 50;
Enter value for price: 1500
old 12:      price := &Price;
new 12:      price := 1500;
Enter value for discount_percentage: 5
old 13:      discount_per := &Discount_Percentage;
new 13:      discount_per := 5;
Product Name      : bat
Quantity          : 50
Price             : 1500
Total Amount      : 75000
Discount (Rs.)    : 3750
Net Amount        : 71250

PL/SQL procedure successfully completed.
```

3. Write a PL/SQL block to calculate the total, percentage and grade of student based on his/her Rollno from RESULT table. (Create RESULT table with Rollno, Name, Sub1, Sub2, Sub3, Sub4, Sub5, Total, Per, Grade attributes with appropriate data type).

-- Write a program to calculate grade

```
CREATE TABLE RESULT1 (  
    Rollno NUMBER(5) PRIMARY KEY,  
    Name VARCHAR2(50),  
    sub1 NUMBER(3) CHECK (sub1 BETWEEN 0 AND 100),  
    sub2 NUMBER(3) CHECK (sub2 BETWEEN 0 AND 100),  
    sub3 NUMBER(3) CHECK (sub3 BETWEEN 0 AND 100),  
    sub4 NUMBER(3) CHECK (sub4 BETWEEN 0 AND 100),  
    sub5 NUMBER(3) CHECK (sub5 BETWEEN 0 AND 100),  
    Total NUMBER(5),  
    Per NUMBER(5,2),  
    Grade VARCHAR2(2)  
);
```

```
INSERT INTO RESULT1 (Rollno, Name, sub1, sub2, sub3, sub4, sub5)  
VALUES (101, 'Harsh', 85, 90, 78, 88, 92);
```

```
INSERT INTO RESULT1 (Rollno, Name, sub1, sub2, sub3, sub4, sub5)  
VALUES (102, 'Rahul', 60, 55, 70, 65, 58);
```

```
INSERT INTO RESULT1 (Rollno, Name, sub1, sub2, sub3, sub4, sub5)  
VALUES (103, 'Priya', 95, 96, 98, 94, 97);
```

```
INSERT INTO RESULT1 (Rollno, Name, sub1, sub2, sub3, sub4, sub5)  
VALUES (104, 'Amit', 45, 52, 48, 50, 55);
```

```
INSERT INTO RESULT1 (Rollno, Name, sub1, sub2, sub3, sub4, sub5)  
VALUES (105, 'Neha', 72, 68, 75, 70, 74);
```

```
INSERT INTO RESULT1 (Rollno, Name, sub1, sub2, sub3, sub4, sub5)  
VALUES (106, 'Karan', 30, 65, 72, 80, 69);
```

```
INSERT INTO RESULT1 (Rollno, Name, sub1, sub2, sub3, sub4, sub5)
VALUES (107, 'Sneha', 88, 84, 91, 86, 90);
```

```
INSERT INTO RESULT1 (Rollno, Name, sub1, sub2, sub3, sub4, sub5)
VALUES (108, 'Riya', 50, 47, 55, 60, 52);
```

```
COMMIT;
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  rno RESULT1.Rollno%TYPE;
  sname RESULT1.Name%TYPE;
  s1 RESULT1.sub1%TYPE;
  s2 RESULT1.sub2%TYPE;
  s3 RESULT1.sub3%TYPE;
  s4 RESULT1.sub4%TYPE;
  s5 RESULT1.sub5%TYPE;
  total RESULT1.Total%TYPE;
  per RESULT1.Per%TYPE;
  grade RESULT1.Grade%TYPE;
```

```
BEGIN
```

```
  rno := &Enter_Roll_No;
```

```
  SELECT Name, sub1, sub2, sub3, sub4, sub5
  INTO sname, s1, s2, s3, s4, s5
  FROM RESULT1
  WHERE Rollno = rno;
```

```
  total := s1 + s2 + s3 + s4 + s5;
  per := total / 5;
```

```
  IF s1 < 35 OR s2 < 35 OR s3 < 35 OR s4 < 35 OR s5 < 35 THEN
    grade := 'F';
    DBMS_OUTPUT.PUT_LINE('RESULT : FAIL');
```

ELSE

IF per >= 90 THEN

grade := 'A+';

ELSIF per >= 80 THEN

grade := 'A';

ELSIF per >= 70 THEN

grade := 'B';

ELSIF per >= 60 THEN

grade := 'C';

ELSIF per >= 50 THEN

grade := 'D';

ELSE

grade := 'F';

END IF;

DBMS_OUTPUT.PUT_LINE('RESULT : PASS');

END IF;

DBMS_OUTPUT.PUT_LINE('-----');

DBMS_OUTPUT.PUT_LINE('Roll No : ' || rno);

DBMS_OUTPUT.PUT_LINE('Name : ' || sname);

DBMS_OUTPUT.PUT_LINE('Total : ' || total);

DBMS_OUTPUT.PUT_LINE('Percentage : ' || per || '%');

DBMS_OUTPUT.PUT_LINE('Grade : ' || grade);

DBMS_OUTPUT.PUT_LINE('-----');

END;

/

Output:

```
Enter value for enter_roll_no: 107
old 14:      rno := &Enter_Roll_No;
new 14:      rno := 107;
RESULT : PASS
-----
Roll No      : 107
Name         : Sneha
Total        : 439
Percentage   : 87.8%
Grade        : A
-----

PL/SQL procedure successfully completed.
```

4.

4.1. Write a program that prints value 1 to 100 numbers using FOR LOOP.

4.2. Write a program that prints value 1 to 100 number using LOOP Command.

4.3. Write a program that prints value 1 to 100 number using WHILE LOOP.

4.1 Program to print numbers 1 to 100 using FOR loop

```
//DECLARE
// i NUMBER;
BEGIN
  FOR i IN 1..100 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
/
```

4.2 Program to print numbers 1 to 100 using LOOP command

```
DECLARE
  i NUMBER := 1;
BEGIN
  LOOP
    DBMS_OUTPUT.PUT_LINE(i);
    i := i + 1;
    EXIT WHEN i > 100;
  END LOOP;
END;
/
```

4.3 Program to print numbers 1 to 100 using WHILE loop

```
DECLARE
  i NUMBER := 1;
BEGIN
  WHILE i <= 100 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
    i := i + 1;
  END LOOP;
END;
```

Output:

1	42	
2	43	
3	44	
4	45	
5	46	
6	47	
7	48	
8	49	
9	50	
10	51	
11	52	
12	53	
13	54	
14	55	
15	56	
16	57	
17	58	
18	59	81
19	60	82
20	61	83
21	62	84
22	63	85
23	64	86
24	65	87
25	66	88
26	67	89
27	68	90
28	69	91
29	70	92
30	71	93
31	72	94
32	73	95
33	74	96
34	75	97
35	76	98
36	77	99
37	78	100
38	79	
39	80	
40	81	
41		

5. Write a PL/SQL block which displays gross salary of employees as per user input EID. (Consider EMP table with EID, EName, Deptno, Deptname Gender, Age, BasicSal) with appropriate data types.)
Gross_Salary: BASICSAL + (DA + HRA + Medical) – PF. Rules: HRA = 15% of basic, DA = 50% of basic, Medical = Rs. 500, PF = 10% of basic.

```
CREATE TABLE EMP(  
  EID NUMBER(3) PRIMARY KEY,  
  EName VARCHAR(15),  
  Deptno NUMBER(3),  
  Deptname VARCHAR(30),  
  Gender VARCHAR(5),  
  Age NUMBER(3),  
  BasicSal NUMBER(10)  
);
```

```
INSERT INTO EMP (EID,EName,Deptno,Deptname,Gender,Age,BasicSal)  
VALUES (101,'Ronak',10,'Loan Department','Male',21,50000);
```

```
INSERT INTO EMP (EID,EName,Deptno,Deptname,Gender,Age,BasicSal)  
VALUES (102,'Hit',20,'Finance Department','Male',20,75000);
```

```
INSERT INTO EMP (EID,EName,Deptno,Deptname,Gender,Age,BasicSal)  
VALUES (103,'Pranav',30,'HR Department','Male',22,45000);
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE  
  id EMP.EID%TYPE;  
  name EMP.EName%TYPE;  
  basic EMP.BasicSal %TYPE;  
  
  hra    NUMBER;  
  da     NUMBER;  
  medical NUMBER := 500;  
  pf     NUMBER;
```

```
gross    NUMBER;

BEGIN

-- Take Employee ID from user
id := &Enter_id;

-- Fetch employee details

SELECT EName, Basicsal
INTO name, basic
FROM EMP
WHERE EID = id;

-- Calculate salary components
hra := basic * 0.15;
da  := basic * 0.50;
pf  := basic * 0.10;

-- Calculate Gross Salary
gross := basic + da + hra + medical - pf;

-- Display Result --
DBMS_OUTPUT.PUT_LINE('-----');
DBMS_OUTPUT.PUT_LINE('Employee ID   : ' || id);
DBMS_OUTPUT.PUT_LINE('Employee Name : ' || name);
DBMS_OUTPUT.PUT_LINE('Basic Salary : ' || basic);
DBMS_OUTPUT.PUT_LINE('DA          : ' || da);
DBMS_OUTPUT.PUT_LINE('HRA         : ' || hra);
DBMS_OUTPUT.PUT_LINE('Medical     : ' || medical);
DBMS_OUTPUT.PUT_LINE('PF          : ' || pf);
DBMS_OUTPUT.PUT_LINE('Gross Salary : ' || gross);
DBMS_OUTPUT.PUT_LINE('-----');

EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Employee ID not found.');
```

END;
/

Output:

```
Enter value for enter_id: 102
old 15:      id := &Enter_id;
new 15:      id := 102;
-----
Employee ID   : 102
Employee Name : Hit
Basic Salary  : 75000
DA            : 37500
HRA          : 11250
Medical       : 500
PF           : 7500
Gross Salary  : 116750
-----
PL/SQL procedure successfully completed.
```

6. Write a PL/SQL block which accepts measurement in feet and displays it in cm, inch and meter.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    feet REAL;
```

```
    inch REAL;
```

```
    cm REAL;
```

```
    meter REAL;
```

```
BEGIN
```

```
    -- Accept input
```

```
    feet := &feet;
```

```
    -- Conversion
```

```
    inch := feet * 12;
```

```
    cm := feet * 30.48;
```

```
    meter := feet * 0.3048;
```

```
    -- Display results
```

```
    DBMS_OUTPUT.PUT_LINE('Measurement in Feet : ' || feet);
```

```
    DBMS_OUTPUT.PUT_LINE('Measurement in Inches : ' || inch);
```

```
    DBMS_OUTPUT.PUT_LINE('Measurement in CM : ' || cm);
```

```
    DBMS_OUTPUT.PUT_LINE('Measurement in Meters : ' || meter);
```

```
END;
```

```
/
```

Output:

```
Enter value for feet: 15
old 8:      feet := &feet;
new 8:      feet := 15;
Measurement in Feet   : 15
Measurement in Inches : 180
Measurement in CM     : 457.2
Measurement in Meters : 4.572
```

```
PL/SQL procedure successfully completed.
```

7. Write a PL/SQL block which converts temperature from Celsius to Fahrenheit.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    celsius    NUMBER;
```

```
    fahrenheit NUMBER;
```

```
BEGIN
```

```
    -- Accept input
```

```
    celsius := &celsius;
```

```
    -- Convert Celsius to Fahrenheit
```

```
    fahrenheit := (celsius * 9 / 5) + 32;
```

```
    -- Display result
```

```
    DBMS_OUTPUT.PUT_LINE('Celsius   = ' || celsius);
```

```
    DBMS_OUTPUT.PUT_LINE('Fahrenheit = ' || fahrenheit);
```

```
END;
```

```
/
```

Output:

```
Enter value for celsius: 32
old   6:      celsius := &celsius;
new   6:      celsius := 32;
Celsius   = 32
Fahrenheit = 89.6

PL/SQL procedure successfully completed.
```

8. Write a PL/SQL block to delete the record of employee for given EID.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    EID NUMBER;
```

```
BEGIN
```

```
    -- Accept Employee ID
```

```
    EID := &EID;
```

```
    -- Delete record
```

```
    DELETE FROM EMP
```

```
    WHERE EID = EID;
```

```
    -- Check if record is deleted
```

```
    IF SQL%ROWCOUNT > 0 THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Employee record deleted successfully.');
```

```
    ELSE
```

```
        DBMS_OUTPUT.PUT_LINE('Employee ID not found.');
```

```
    END IF;
```

```
    COMMIT;
```

```
END;
```

```
/
```

Output:

```
Enter value for eid: 101
old   5:      EID := &EID;
new   5:      EID := 101;
Employee record deleted successfully.

PL/SQL procedure successfully completed.
```

9. Write a PL/SQL block that calculates the simple interest based on given principal amount, rate of interest and number of years.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
  p NUMBER;
```

```
  r NUMBER;
```

```
  n NUMBER;
```

```
  si NUMBER;
```

```
BEGIN
```

```
  -- Accept inputs
```

```
  p := &principal;
```

```
  r := &rate;
```

```
  n := &years;
```

```
  -- Calculate Simple Interest
```

```
  si := (p * r * n) / 100;
```

```
  -- Display result
```

```
  DBMS_OUTPUT.PUT_LINE('Simple Interest = ' || si);
```

```
END;
```

```
/
```

Output:

```
Enter value for principal: 10000
old   8:      p := &principal;
new   8:      p := 10000;
Enter value for rate: 10
old   9:      r := &rate;
new   9:      r := 10;
Enter value for years: 1
old  10:      n := &years;
new  10:      n := 1;
Simple Interest = 1000

PL/SQL procedure successfully completed.
```

10. Write a PL/SQL block to calculate the square and cube of the given number.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    num  NUMBER;
```

```
    square NUMBER;
```

```
    cube  NUMBER;
```

```
BEGIN
```

```
    -- Accept input
```

```
    num := &num;
```

```
    -- Calculate square and cube
```

```
    square := num * num;
```

```
    cube := num * num * num;
```

```
    -- Display result
```

```
    DBMS_OUTPUT.PUT_LINE('Square = ' || square);
```

```
    DBMS_OUTPUT.PUT_LINE('Cube = ' || cube);
```

```
END;
```

```
/
```

Output:

```
Enter value for num: 15
old   7:      num := &num;
new   7:      num := 15;
Square = 225
Cube = 3375

PL/SQL procedure successfully completed.
```


10. Write a simple procedure that increases by the salary of employees for the given department no. by percentage inputted by the user using IN parameter. Also, handle the exception if inputted department number not found.

```
CREATE TABLE EMP1(  
  EID NUMBER(3) PRIMARY KEY,  
  EName VARCHAR(15),  
  Deptno NUMBER(3),  
  Deptname VARCHAR(30),  
  Gender VARCHAR(5),  
  Age NUMBER(3),  
  BasicSal NUMBER(10)  
);
```

```
INSERT INTO EMP1 (EID,EName,Deptno,Deptname,Gender,Age,BasicSal)  
VALUES (101,'Ronak',10,'Loan Department','Male',21,50000);
```

```
INSERT INTO EMP1 (EID,EName,Deptno,Deptname,Gender,Age,BasicSal)  
VALUES (102,'Hit',20,'Finance Department','Male',20,75000);
```

```
INSERT INTO EMP1 (EID,EName,Deptno,Deptname,Gender,Age,BasicSal)  
VALUES (103,'Pranav',30,'HR Department','Male',22,45000);
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE  
  id EMP1.EID%TYPE;  
  name EMP1.EName%TYPE;  
  basic EMP1.BasicSal %TYPE;
```

```
hra    NUMBER;  
da     NUMBER;  
medical NUMBER := 500;  
pf     NUMBER;  
gross  NUMBER;
```

BEGIN

-- Take Employee ID from user

id := &Enter_id;

-- Fetch employee details

SELECT EName, Basicsal

INTO name, basic

FROM EMP1

WHERE EID = id;

-- Calculate salary components

hra := basic * 0.15;

da := basic * 0.50;

pf := basic * 0.10;

-- Calculate Gross Salary

gross := basic + da + hra + medical - pf;

-- Display Result --

DBMS_OUTPUT.PUT_LINE('-----');

DBMS_OUTPUT.PUT_LINE('Employee ID : ' || id);

DBMS_OUTPUT.PUT_LINE('Employee Name : ' || name);

DBMS_OUTPUT.PUT_LINE('Basic Salary : ' || basic);

DBMS_OUTPUT.PUT_LINE('DA : ' || da);

DBMS_OUTPUT.PUT_LINE('HRA : ' || hra);

DBMS_OUTPUT.PUT_LINE('Medical : ' || medical);

DBMS_OUTPUT.PUT_LINE('PF : ' || pf);

DBMS_OUTPUT.PUT_LINE('Gross Salary : ' || gross);

DBMS_OUTPUT.PUT_LINE('-----');

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Employee ID not found.');

END;

/

SET SERVEROUTPUT ON;

```
CREATE OR REPLACE PROCEDURE inc_salary(  
    deptno IN NUMBER,  
    percent IN NUMBER  
)  
IS  
BEGIN  
    UPDATE EMP1  
    SET BasicSal= BasicSal+ (BasicSal * percent / 100)  
    WHERE Deptno = deptno;  
  
    IF SQL%ROWCOUNT = 0 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Department not found');  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');    END IF;  
  
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE(SQLERRM);  
END;  
/
```

----Execute the Procedure----

```
BEGIN  
    inc_salary(30, 10); -- Increase salary by 10% for Department 10  
END;  
/
```

Output:

```
Enter value for enter_id: 102
old 15:      id := &Enter_id;
new 15:      id := 102;
-----
Employee ID   : 102
Employee Name : Hit
Basic Salary  : 75000
DA            : 37500
HRA           : 11250
Medical       : 500
PF            : 7500
Gross Salary  : 116750
-----
PL/SQL procedure successfully completed.
```

```
SQL> select * from emp1;

      EID ENAME      DEPTNO DEPTNAME      GENDE
-----
      AGE  BASICSAL
-----
      101 Ronak      10 Loan Department      Male
      21    55000
      102 Hit        20 Finance Department    Male
      20    82500
      103 Pranav     30 HR Department         Male
      22    49500
```

12. Write a procedure that search whether the given employee number is present or not in the table if present then print name, salary and department number of that employee. (Use both IN and OUT mode variables) and Write a PL/SQL block to call the SEARCH_EMP procedure.

```
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE PROCEDURE SEARCH_EMP(
```

```
    P_EMP_ID  IN EMP1.EID%TYPE,  
    P_EMP_NAME OUT EMP1.ENAME%TYPE,  
    P_EMP_SAL OUT EMP1.BASICSAL%TYPE,  
    P_EMP_DNO OUT EMP1.DEPTNO%TYPE
```

```
)
```

```
AS
```

```
BEGIN
```

```
    SELECT ENAME, BASICSAL, DEPTNO  
    INTO P_EMP_NAME, P_EMP_SAL, P_EMP_DNO  
    FROM EMP1  
    WHERE EID = P_EMP_ID;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE('EMPLOYEE ID ' || P_EMP_ID || ' NOT FOUND');
```

```
END;
```

```
/
```

```
DECLARE
```

```
    G_NAME EMP1.ENAME%TYPE;
```

```
    G_SAL EMP1.BASICSAL%TYPE;
```

```
    G_DNO EMP1.DEPTNO%TYPE;
```

```
BEGIN
```

```
    SEARCH_EMP(102, G_NAME, G_SAL, G_DNO);
```

```
    DBMS_OUTPUT.PUT_LINE('EMPLOYEE NAME: ' || G_NAME);
```

```
    DBMS_OUTPUT.PUT_LINE('BASIC SALARY: ' || G_SAL);
```

```
        DBMS_OUTPUT.PUT_LINE('DEPARTMENT NO: ' || G_DNO);  
END;  
/
```

Output:

```
EMPLOYEE NAME: Hit  
BASIC SALARY: 90750  
DEPARTMENT NO: 20
```

```
PL/SQL procedure successfully completed.
```

13. Write a procedure that accept employee ID and remove records from employee table. Also, create a calling program to call procedure.

```
CREATE OR REPLACE PROCEDURE DELETE_EMP(
EMP_ID IN EMP1.EID%TYPE
)AS

BEGIN
    DELETE FROM EMP1
    WHERE EID=EMP_ID;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(TO_CHAR(EMP_ID) || ' NO DATA FOND ');
END;
/

SET SERVEROUTPUT ON;

DECLARE
    EID EMP1.EID%TYPE;
BEGIN
    EID := &EMPLOYEE_ID;
    DELETE_EMP(EID);
END;
/
```

Output:

```
Enter value for employee_id: 102
old 4:      EID := &EMPLOYEE_ID;
new 4:      EID := 102;

PL/SQL procedure successfully completed.

SQL> select * from emp1;

      EID ENAME          DEPTNO DEPTNAME          GENDE
-----
      AGE  BASICSAL
-----
      101 Ronak          10 Loan Department      Male
      21      60500
      103 Pranav          30 HR Department       Male
      22      54450
```

14. Write a function that returns the age category of an employee.

Categories:

Age < 18 -> Minor

18 to 60 -> Adult

Age > 60 -> Senior

```
CREATE OR REPLACE FUNCTION GET_AGE_CATEGORY (  
    P_AGE IN NUMBER  
) RETURN VARCHAR2  
IS  
    V_CATEGORY VARCHAR2(20);  
BEGIN  
    IF P_AGE IS NULL OR P_AGE < 0 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'INVALID AGE.');    END IF;  
  
    IF P_AGE < 18 THEN  
        V_CATEGORY := 'MINOR';  
    ELSIF P_AGE BETWEEN 18 AND 59 THEN  
        V_CATEGORY := 'ADULT';  
    ELSE  
        V_CATEGORY := 'SENIOR';  
    END IF;  
  
    RETURN V_CATEGORY;  
END;  
/  
  
BEGIN  
    DBMS_OUTPUT.PUT_LINE(GET_AGE_CATEGORY(15));  
    DBMS_OUTPUT.PUT_LINE(GET_AGE_CATEGORY(30));  
    DBMS_OUTPUT.PUT_LINE(GET_AGE_CATEGORY(65));  
END;  
/
```


Output:

```
SQL>
SQL>
SQL> BEGIN
  2      DBMS_OUTPUT.PUT_LINE(GET_AGE_CATEGORY(15));
  3      DBMS_OUTPUT.PUT_LINE(GET_AGE_CATEGORY(30));
  4      DBMS_OUTPUT.PUT_LINE(GET_AGE_CATEGORY(65));
  5  END;
  6  /
MINOR
ADULT
SENIOR

PL/SQL procedure successfully completed.
```

15. Write a PACKAGE which includes Package Declaration, Package Body and call the individual object from the Package.

-- 1. PACKAGE SPECIFICATION (DECLARATION)

```
CREATE OR REPLACE PACKAGE MATH_OPERATIONS AS
    PROCEDURE ADD_NUMBERS(P_NUM1 IN NUMBER, P_NUM2 IN NUMBER);
    FUNCTION MULTIPLY_NUMBERS(P_NUM1 IN NUMBER, P_NUM2 IN NUMBER)
RETURN NUMBER;
    G_LAST_RESULT NUMBER;
END MATH_OPERATIONS;
/
```

-- 2. PACKAGE BODY (IMPLEMENTATION)

```
CREATE OR REPLACE PACKAGE BODY MATH_OPERATIONS AS
    PROCEDURE ADD_NUMBERS(P_NUM1 IN NUMBER, P_NUM2 IN NUMBER) IS
    BEGIN
        G_LAST_RESULT := P_NUM1 + P_NUM2;
        DBMS_OUTPUT.PUT_LINE('SUM: ' || G_LAST_RESULT);
    END ADD_NUMBERS;
    FUNCTION MULTIPLY_NUMBERS(P_NUM1 IN NUMBER, P_NUM2 IN NUMBER)
RETURN NUMBER IS
    BEGIN
        G_LAST_RESULT := P_NUM1 * P_NUM2;
        RETURN G_LAST_RESULT;
    END MULTIPLY_NUMBERS;

END MATH_OPERATIONS;
/
```

-- 3. CALLING PACKAGE OBJECTS

SET SERVEROUTPUT ON;

BEGIN

MATH_OPERATIONS.ADD_NUMBERS(10, 20);

```
DBMS_OUTPUT.PUT_LINE('PRODUCT: ' ||  
MATH_OPERATIONS.MULTIPLY_NUMBERS(5, 6));  
DBMS_OUTPUT.PUT_LINE('LAST RESULT STORED IN G_LAST_RESULT: ' ||  
MATH_OPERATIONS.G_LAST_RESULT);  
END;  
/
```

Output:

```
SQL> SET SERVEROUTPUT ON;  
SQL>  
SQL> BEGIN  
2     MATH_OPERATIONS.ADD_NUMBERS(10, 20);  
3     DBMS_OUTPUT.PUT_LINE('PRODUCT: ' || MATH_OPERATIONS.MULTIPLY_NUMBERS(5, 6));  
4     DBMS_OUTPUT.PUT_LINE('LAST RESULT STORED IN G_LAST_RESULT: ' || MATH_OPERATIONS.G_LAST_RESULT);  
5 END;  
6 /  
SUM: 30  
PRODUCT: 30  
LAST RESULT STORED IN G_LAST_RESULT: 30  
  
PL/SQL procedure successfully completed.
```